RUSTIKON
Powered by SoftwareMill

exe1n

Running code 0 times / second for maximum performance

Wojciech Brożek

RUSTIKON
Powered by SoftwareMill

exe1n

github.com/Niedzwiedzw
wojciech.brozek@niedzwiedz.it
exein.io

```
trait Exein: Rust + Security {}
```

60FPS

60FPS (~16ms per frame)

Solution: doing less at runtime!

Solution: doing less at runtime!

(some nightly features)

Solution: doing less at runtime!

(some nightly features)

(some unsafe code)

```rust
/// we need a way to push/pop inside `const {}` blocks
struct ConstVec<T> {
    // ..
}
```

```rust
const MAX_SIZE: usize = 2usize.pow(13);

/// we need a way to push/pop inside `const {}` blocks
pub struct ConstVec<T> {
    memory: [MaybeUninit<T>; MAX_SIZE],
    len: usize,
}
```

```rust
impl<T> ConstVec<T> {
  pub const fn new() -> Self {
      Self {
          memory: unsafe {
            MaybeUninit::uninit().assume_init()
          },
          len: 0,
      }
   }
}
```

```rust
impl<T> ConstVec<T> {
    pub const fn push(self, value: T) -> Self {
        let mut memory = self.memory;
        memory[self.len] = MaybeUninit::new(value);
        Self { memory, len: self.len + 1 }
    }
}
```

```rust
impl<T> ConstVec<T> {
    pub const fn as_ref(&self) -> &[T] {
        unsafe {
            &*(
                self.memory.split_at(self.len).0
                as *const [MaybeUninit<T>]
                as *const [T]
            )
        }
    }
}
```

```rust
impl ConstVec<u8> {
    pub const fn push_str(self, s: &'static str) -> Self {
        let mut this = self;
        let mut extend = s.as_bytes();
        while let [next, tail @ ..] = extend {
            this = this.push(*next);
            extend = tail;
        }
        this
    }
}
```

```rust
impl ConstVec<u8> {
    pub const fn as_str(&'static self) -> &'static str {
        unsafe {
            std::str::from_utf8_unchecked(self.as_ref())
        }
    }
}

// We can build strings at compile time
```

HTML builder… in Rust’s type system?

```
// ./src/lib.rs
// + #![feature(adt_const_params)]
// + #![feature(unsized_const_params)]
```

```rust
pub trait IsAttribute {
    const ATTRIBUTE_BYTES: ConstVec<u8>;
    const ATTRIBUTE: &'static str;
// }

pub trait IsChild {
    const CHILD_BYTES: ConstVec<u8>;
    const CHILD: &'static str;
}
```

```rust
pub trait IsChildren {
    const CHILDREN_BYTES: ConstVec<u8>;
    const CHILDREN: &'static str;
}
pub trait IsAttributes {
    const ATTRIBUTES_BYTES: ConstVec<u8>;
    const ATTRIBUTES: &'static str;
}
```

```rust
/// For example: "background: black"
pub struct Attribute<
    const K: &'static str,
    const V: &'static str,
>;
```

```rust
impl<const K: &'static str, const V: &'static str>
    IsAttribute for Attribute<K, V> {
    const ATTRIBUTE_BYTES: ConstVec<u8> = const {
        ConstVec::new()
            .push_str(K).push_str(r#"="#)
            .push_str(V).push_str(r#""#)
    };
    // ...
}
```

```rust
impl<const K: &'static str, const V: &'static str>
    IsAttribute for Attribute<K, V> {
    // ...
    const ATTRIBUTES: &'static str = const {
        Self::ATTRIBUTES_BYTES.as_str()
    };
}
```

```rust
pub struct PushAttribute<
    T: IsAttributes,
    Attribute: IsAttribute,
>(PhantomData<(T, Attribute)>);
```

```rust
impl<T: IsAttributes, A: IsAttribute>
    IsAttributes for PushAttribute<T, A> {
    const ATTRIBUTES_BYTES: ConstVec<u8> = const {
        ConstVec::new()
            .push_str(T::ATTRIBUTES)
            .push_str(" ")
            .push_str(A::ATTRIBUTE)
    };
}
```

```rust
impl<const TAG: &'static str, A, C> DomTree<TAG, A, C>
where
    A: IsAttributes + 'static,
    C: IsChildren + 'static,
{
    pub const fn child<Child: IsChild + 'static>(
        self,
        _child: Finished<Child>,
    ) -> DomTree<T, Attributes, PushChild<C, Child>> {
        DomTree::new()
    }
```